

Fullstack JavaScript Project Guide

Pocket Ledger

A beginner-friendly walkthrough of the personal finance dashboard

Prepared on May 25, 2026

How to Use This Guide

This guide explains Pocket Ledger as a learning project. It shows what each part of the app does, why the files exist, and how a user action moves through the browser, server, and data file.

Best way to study: Keep the project open in WebStorm while reading. When a section mentions a file, open that file and trace the code yourself.

Suggested study loop:

- Run the app with **npm run dev**.
- Create an account in the browser.
- Add one expense and watch the dashboard change.
- Find the matching frontend and backend code.
- Change one small thing, test it, and repeat.

Project Overview

Pocket Ledger is a fullstack JavaScript app for tracking personal expenses. Users can register, log in, create expenses, edit them, delete them, filter by month or category, and view simple charts.

Layer	Technology	What it does
Frontend	React + Vite	Shows pages, forms, tables, filters, and charts.
Routing	React Router	Chooses login, register, or dashboard pages.
HTTP Client	Axios	Sends requests from the browser to the API.
Backend	Node.js + Express	Receives API requests and returns JSON.
Authentication	bcryptjs + JWT	Hashes passwords and keeps users signed in.
Storage	JSON file	Stores users and expenses locally for learning.
Charts	Recharts	Draws pie and bar charts from summary arrays.

Mental Model: Adding an Expense

Browser form

- > React state stores typed values
- > Axios sends POST /api/expenses
- > Express route checks the JWT
- > Validator checks amount, date, category, description
- > JSON database file is updated
- > Backend returns the new expense
- > Dashboard reloads expenses and summary
- > Table and charts update

This chain is the heart of fullstack programming. A button click in the browser causes code to run in several places: the component, the API client, the route, the database helper, and then the component again.

Folder Structure

Path	Purpose
package.json	Root workspace scripts. Runs client and server together.
client/	Everything that runs in the browser.
client/src/pages/	Top-level page components: Login, Register, Dashboard.
client/src/services/	Shared frontend helpers for API calls and authentication state.
client/src/styles/app.css	Main styling for layout, forms, tables, and responsive behavior.
server/	Everything that runs in Node.js.
server/src/index.js	Creates the Express app and starts the API server.
server/src/routes.js	Defines the API endpoints.
server/src/auth.js	Creates and verifies login tokens.
server/src/db.js	Reads and writes the local JSON data file.
server/src/validators.js	Checks expense data before saving.

Running the Project

Install dependencies once:

```
npm install
```

Start both the frontend and backend:

```
npm run dev
```

The React app opens at **http://127.0.0.1:5173**. The backend API listens on **http://localhost:4000**. Vite proxies browser requests from **/api** to the backend server.

Common beginner mistake: Do not open client/index.html directly in the browser. Vite needs to run so React modules, hot reload, and the API proxy work correctly.

Frontend Walkthrough

main.jsx

This entry file finds the root element in index.html and tells React to render the app inside it. BrowserRouter enables navigation, and AuthProvider makes login state available to every component.

```
createRoot(rootElement).render(  
  <BrowserRouter>  
    <AuthProvider>  
      <App />  
    </AuthProvider>  
  </BrowserRouter>  
);
```

App.jsx

App.jsx decides which page the user sees. PrivateRoute protects the dashboard from visitors who are not logged in.

- The /login route shows the login page.
- The /register route shows the account creation page.
- The / route shows the dashboard after authentication.

AuthContext.jsx

The authentication context solves a common React problem: many components need to know who is logged in. Instead of passing user data through every component manually, the context stores it in one shared place.

Function	Job
loadUser	Checks localStorage for a token and asks the API who the user is.
login	Sends credentials to the backend, stores the token, and updates user state.
register	Creates an account and signs the user in immediately.
logout	Removes the token and clears the user state.

Dashboard.jsx

Dashboard.jsx owns the main workflow: filters, form state, editing state, expense data, summary data, and chart rendering.

- useState stores changing screen data such as expenses, selected month, selected category, and form fields.
- useEffect reloads dashboard data whenever the month or category filter changes.
- handleSubmit creates a new expense or updates an existing one.
- startEdit copies an expense into the form.
- removeExpense deletes an expense and reloads dashboard data.

Learning checkpoint: Find handleSubmit in Dashboard.jsx. Notice how one function handles both Add and Edit by checking whether editingId exists.

Backend Walkthrough

index.js

The server entry file creates the Express app, enables JSON request bodies, allows frontend requests with CORS, mounts routes under /api, and starts listening on the configured port.

routes.js

Routes are the public contract of the backend. Each route answers one kind of request.

Endpoint	Method	Login	Purpose
/api/auth/register	POST	No	Create an account.
/api/auth/login	POST	No	Sign in and receive a JWT.
/api/auth/me	GET	Yes	Return the current user.
/api/expenses	GET	Yes	List expenses with optional filters.
/api/expenses	POST	Yes	Create an expense.
/api/expenses/:id	PUT	Yes	Update one expense.
/api/expenses/:id	DELETE	Yes	Delete one expense.
/api/summary	GET	Yes	Return monthly totals for charts.

auth.js

Authentication has two halves. `createToken` signs a JWT after a successful login. `requireAuth` checks the token before protected routes run.

```
const header = req.headers.authorization || '';
const token = header.startsWith('Bearer ') ? header.slice(7) : null;
```

db.js

This project uses a JSON file instead of a real database. That keeps setup simple while still teaching the same pattern: read data, change data, write data back.

Production note: A real finance app should use a proper database, stronger secrets, HTTPS, and more careful security controls.

Data Model

The JSON file has two top-level arrays: `users` and `expenses`.

```
{
  "users": [],
  "expenses": []
}
```

Expense field	Meaning
id	Unique ID for this expense.
userId	Connects the expense to the logged-in user.
amount	Number spent.
date	Date in YYYY-MM-DD format.
category	Bucket such as Food, Rent, Groceries, or Transport.
description	Short label shown in the table.

Expense field	Meaning
notes	Optional extra detail.
createdAt / updatedAt	Timestamps for record history.

CRUD Explained Through This Project

CRUD action	User sees	HTTP request	Backend route
Create	Add expense form	POST /api/expenses	router.post('/expenses')
Read	Expense log and charts	GET /api/expenses, GET /api/summary	router.get(...)
Update	Edit button fills the form	PUT /api/expenses/:id	router.put('/expenses/:id')
Delete	Trash button removes a row	DELETE /api/expenses/:id	router.delete('/expenses/:id')

Most web apps are built from this pattern. A social app creates posts, reads feeds, updates profiles, and deletes comments. A store creates orders, reads products, updates carts, and deletes saved addresses.

How the Charts Work

The frontend asks the backend for a summary. The backend groups expenses by category and by date, then the frontend passes those arrays into Recharts.

```

byCategory: [
  { category: 'Food', total: 25.50 },
  { category: 'Transport', total: 12.00 }
]

byDay: [
  { date: '2026-05-24', total: 37.50 }
]

```

Why this is useful: Charts usually need simple arrays of objects. Once your data has that shape, a charting library can do the drawing for you.

Security Concepts in Plain English

- Passwords are not stored directly. bcrypt turns them into hashes.
- A JWT token proves that the user has already logged in.
- Protected routes use `requireAuth` before reading or changing private data.
- Every expense has a `userId` so one user cannot see another user's entries.
- The JWT secret should be changed before sharing or deploying the app.

Beginner Debugging Guide

Problem	Where to look first	Likely cause
Login does not work	Browser console and server terminal	Wrong credentials or API server not running.
Dashboard is blank	Network tab	Frontend cannot reach <code>/api/auth/me</code> .
Expense will not save	Response body from POST <code>/api/expenses</code>	Validation failed or missing token.
Charts are empty	GET <code>/api/summary</code> response	No expenses for the selected month.
Port already in use	Terminal output	Another dev server is already running.

Practice Exercises

- Add a new category called Subscriptions in both `validators.js` and `Dashboard.jsx`.
- Change the currency formatter from USD to PLN.
- Add a Budget field and show how much is left this month.
- Add a search box that filters expenses by description.
- Replace the JSON file with SQLite or MongoDB.
- Add a confirmation step before deleting an expense.
- Add a line chart that compares category spending across months.

Glossary

Term	Meaning
Component	A reusable piece of React UI.
State	Data that changes while the app runs.
Props	Values passed from one component to another.
Route	A URL path handled by the frontend or backend.
API	A set of endpoints the frontend can call.
Middleware	Code that runs before a route handler.
JWT	A signed token used to identify a logged-in user.
Hash	A one-way transformed password value.
CRUD	Create, Read, Update, Delete.

Suggested Learning Path

Study the app in passes instead of trying to understand everything at once.

- Pass 1: Run the app and use every button.
- Pass 2: Follow the frontend files only.
- Pass 3: Follow the backend files only.
- Pass 4: Trace one request from button click to JSON response.
- Pass 5: Add one small feature without changing the whole project.

Final thought: A fullstack app is a conversation between a browser, a server, and some data. Once you can trace that conversation, you can build much larger projects.